# 第 3 章練習解答　程序

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

## OS03-E01　狀態轉移

1. P1 running 因 I/O 進入 waiting，scheduler 從 ready queue 選 P2，P2 ready 變 running。磁碟完成後 P1 waiting 變 ready。
2. P1 阻塞時可切到 P2；若排程器允許搶占，P1 變 ready 後也可能觸發切換。若 nonpreemptive，P2 可能先繼續到阻塞或讓出。
3. 不必須。P1 只進入 ready，仍要與其他 ready process 比較，由 scheduler 選中且 dispatcher 載入狀態後才 running。

**檢查。** 每個時刻只有一個 running 是單 CPU 模型的檢查條件。

## OS03-E02　Zombie 與 orphan

1. 當子程序已終止，而父程序仍未 wait 收取終止狀態時成為 zombie。只執行 fork 或 exec 本身不會造成 zombie。
2. 父程序先結束而子程序仍存活時，子程序成為 orphan，通常由系統重新指定監護者。
3. 父程序要處理 child exit 並呼叫 wait 類操作。多子程序系統可用訊號或被集中管理的 reaper 收割，避免同步阻塞主要服務。

**檢查。** zombie 是已終止狀態，orphan 是 parent 關係失效。

## OS03-E03　有界message channel與終止

1. full時blocking send J3等到receive釋放slot才入列；nonblocking回would-block且不改queue。send成功僅指入列，不是job effect完成。
2. 本題atomic完整message，不能任意partial。Byte stream則按bytes回傳累積framing，實際pipe另按PIPE_BUF/length/阻塞規則核對，不可混用。
3. 禁止新jobs後依FIFO送STOP，consumer完成之前jobs再取STOP並ack，管理者之後cleanup。2consumers只有1STOP可能使另一個永久等，需每人1STOP或closed/broadcast protocol。原生remove不自帶drain保證。

**檢查。** 所有推理保持同一message契約。

# 第 3 章練習解答　程序 （續）

## OS03-E04　Context與物件身分

1. 先保存P100的40/7/9000到其task相關context，再載入P101的120/3/A000及正確memory context；P100稍後恢復自己的值。ISA決定保存PC所對應完成/重啟邊界。
2. 不能。唯讀code frames可共享但private globals/heap/stack仍獨立；同VA可指不同frame；同fd3可引用不同object。PCB引用資源，並非整個heap都放PCB。
3. 不必。同process entry/return可只換mode；同process threads可換register/stack而共用user AS。Mode/task/AS switch是不同事件。

**檢查。** 核對中間狀態、一般理由與成立前提。

## OS03-E05　Lost wakeup

1. waiter check false→handler done=true但queue空→waiter才enqueue/sleep。沒有第二次completion便永久waiting。
2. waiter在L內check，false則enqueue並設WAITING再unlock，之後只在仍WAITING時睡。handler在L內設done並把waiter改READY。completion早於check則直接見true不睡；持L時handler延後到已登記；已睡後可找到並醒；unlock後schedule前先醒則必須保留READY，不能無條件睡下。
3. 只保證ready資格。還需scheduler選中/dispatcher恢復；nonpreemptive時當前工作可繼續。

**檢查。** 核對中間狀態、一般理由與成立前提。

## OS03-E06　Fork樹與生命週期

1. 第一次P0→P1，第二行P0→P2且P1→P3，總4。每個現有process都到下一次成功fork才有N(k+1)=2N(k)、N0=1，歸納2^k。僅parent續fork、child exec/exit或失敗則不符。
2. O1由0→100→150，child從100起讀。fork複製fd table entries可仍引用同description；另open同inode得O2可有獨立offset0。這與private-memory COW不同。
3. exec換同一process image，不增process。exit7到wait前保留可收取狀態是zombie；parent先退出而child仍活著是orphan關係。wait先辨normal exit再解7，不把原始status word直接當exit7。

**檢查。** 核對中間狀態、一般理由與成立前提。

# 第 3 章練習解答　程序 （續）

## OS03-E07　SPSC ring與publication

1. (0,0)[]→(0,1)[A]→(0,2)[A,B]；嘗試C full不變；getA→(1,2)[B]；putC→(1,0)[B,C]；getB→(2,0)[C]；getC→(0,0)[]。capacity2。
2. 先寫slot再release tail，consumer acquire看見publish後才讀，避免半成品；先讀slot再release head，producer acquire看見釋放後才reuse，避免覆寫未讀資料。full/empty及唯一producer/consumer維持0≤size≤2與FIFO。
3. 不可。C/C++ plain共享衝突可data race；沒有原子/順序保證。2producers可競爭同tail/slot，違反唯一writer。跨process atomics需平台明確支援，不能只憑C++型別保證。

**檢查。** 核對中間狀態、一般理由與成立前提。

## OS03-E08　Stream framing與shared object

1. 第一塊半header不輸出；第二塊length3但僅AB仍不輸出；第三塊補C輸出ABC，接著length2與DE輸出DE。read邊界不是record邊界。
2. header3卻只有AB，EOF為truncation；length9超max8先拒絕，不盲目配置。EOF剛好在完整record邊界可正常結束。
3. 各為100010與700010，仍可指同backing。共享offset比raw pointer可移植；close fd不自動unmap，shm_unlink移名不使既有mapping立即失效，初始化/publication仍需同步。
4. 不等。mq_unlink移名，既有open引用續用至最後close；IPC_RMID移除queue，blocked操作醒來回EIDRM。STOP/drain必須另設application protocol。

**檢查。** 核對中間狀態、一般理由與成立前提。

## OS03-E09　RPC重試與wire encoding

1. 00 00 00 4D | 00 01 | 00 00 00 19，共10bytes。明定width/endianness，不傳native padding或raw pointer。
2. A request未到balance100；B已commit75但reply遺失。Client都只timeout；換新ID重試在B扣到50，不重試在A遺漏。不能由timeout判操作未執行。
3. 以(client identity,id)索引，balance更新與result75在同一可恢復atomic transaction；重送77取75再扣0。並行需transaction/unique constraint，不能都先查無再各扣。
4. RAM重啟遺失；先debit後記id有crash window；清ID後仍准舊重試可重做。需共同持久commit、ID/epoch作用域與保留期。At-most-once仍可零次，不是無條件exactly-once。

**檢查。** 核對中間狀態、一般理由與成立前提。

# 第 3 章練習解答　程序 （續）

## 原創驗算程式附錄: process_ipc_models.py

```
"""Sequential teaching models only; not C++ atomics or persistent database code."""

class Ring:
    """One reserved slot; each method is one abstract atomic operation."""
    def __init__(self, slots=3):
        if slots<2:raise ValueError("at least two physical slots")
        self.slots=[None]*slots
        self.head=self.tail=0

    def put(self,item):
        next_tail=(self.tail+1)%len(self.slots)
        if next_tail==self.head:return False
        self.slots[self.tail]=item
        self.tail=next_tail
        return True

    def get(self):
        if self.head==self.tail:return False,None
        item=self.slots[self.head]
        self.head=(self.head+1)%len(self.slots)
        return True,item

    def pending(self):
        result=[];cursor=self.head
        while cursor!=self.tail:
            result.append(self.slots[cursor])
            cursor=(cursor+1)%len(self.slots)
        return result


class RecordParser:
    def __init__(self,maximum=8):
        self.buffer=bytearray();self.maximum=maximum

    def feed(self,fragment):
        self.buffer.extend(fragment)
        records=[]
        while len(self.buffer)>=2:
            length=int.from_bytes(self.buffer[:2],"big")
            if length>self.maximum:raise ValueError("record length exceeds limit")
            if len(self.buffer)<2+length:break
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 第 3 章練習解答　程序 （續）

```
            records.append(bytes(self.buffer[2:2+length]))
            del self.buffer[:2+length]
        return records

    def finish(self):
        if self.buffer:raise ValueError("truncated record at EOF")


def debit_transaction(state,client,request_id,amount):
    """Return one indivisible next-state proposal: balance and dedup record."""
    balance,records=state
    key=(client,request_id)
    if key in records:
        original,result=records[key]
        if original!=amount:raise ValueError("request ID reused for another operation")
        return state,result
    if amount<=0 or amount>balance:raise ValueError("invalid debit")
    after=balance-amount
    copied=dict(records);copied[key]=(amount,after)
    return (after,copied),after


if __name__=="__main__":
    ring=Ring()
    for item in "AB":assert ring.put(item)
    assert not ring.put("C")
    print("full:",ring.head,ring.tail,ring.pending())
    assert ring.get()==(True,"A") and ring.put("C")
    print("wrapped:",ring.head,ring.tail,ring.pending())
    parser=RecordParser();messages=[]
    for fragment in [bytes.fromhex("00"),bytes.fromhex("03 41 42"),bytes.fromhex("43 00 02 44
45")]:
        messages.extend(parser.feed(fragment))
    parser.finish();print("records:",messages)
    state=(100,{})
    state,result=debit_transaction(state,"A",77,25)
    state,repeated=debit_transaction(state,"A",77,25)
    assert result==repeated==75 and state[0]==75
    print("same-ID retry:",state,result,repeated)
```

**參考資料與署名**

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study